

NeetCode 150 Approach Guide - Problem -> Approach Ladder

Use each line as a quick revision path: start with brute force, then move to the optimized pattern.

Arrays & Hashing

- 001. LC 217: Contains Duplicate - Brute force compare pairs -> sort and check neighbors -> hash set in one pass.
- 002. LC 242: Valid Anagram - Sort both strings -> frequency array/hash map counts -> compare counts.
- 003. LC 1: Two Sum - Brute force all pairs -> sort + two pointers (if indices handled) -> hash map complement.
- 004. LC 49: Group Anagrams - Pairwise compare sorted words -> key by sorted string -> key by 26-count tuple.
- 005. LC 347: Top K Frequent Elements - Count then sort by freq -> min-heap size k -> bucket sort by frequency.
- 006. LC 271: Encode and Decode Strings - Join with delimiter (unsafe) -> length-prefix each string, decode by reading length then content.
- 007. LC 238: Product of Array Except Self - Nested products -> prefix/suffix arrays -> two passes with O(1) extra output.
- 008. LC 36: Valid Sudoku - Check each row/col/box separately -> sets for rows, cols, boxes in one scan.
- 009. LC 128: Longest Consecutive Sequence - Sort and count streaks -> hash set, start only at numbers with no predecessor.

Two Pointers

- 010. LC 125: Valid Palindrome - Clean string then compare -> two pointers skipping non-alphanumeric.
- 011. LC 167: Two Sum II - Input Array Is Sorted - Brute force -> binary search complement -> left/right two pointers.
- 012. LC 15: 3Sum - Brute force triples -> sort + two-sum two pointers, skip duplicates.
- 013. LC 11: Container With Most Water - Brute force areas -> two pointers, move shorter height inward.
- 014. LC 42: Trapping Rain Water - For each bar max left/right -> prefix/suffix max -> two pointers with running max.

Sliding Window

- 015. LC 121: Best Time to Buy and Sell Stock - Brute force buy/sell pairs -> one pass track min price and best profit.
- 016. LC 3: Longest Substring Without Repeating Characters - Brute substrings -> sliding window set -> map last seen index and jump left.
- 017. LC 424: Longest Repeating Character Replacement - Check all windows -> sliding window with counts and max frequency.
- 018. LC 567: Permutation in String - Sort each window -> sliding window char counts compare to target.
- 019. LC 76: Minimum Window Substring - Brute all substrings -> expand/right until valid, shrink/left while valid.
- 020. LC 239: Sliding Window Maximum - Brute max each window -> max-heap with lazy removal -> monotonic deque.

Stack

- 021. LC 20: Valid Parentheses - Repeatedly remove pairs -> stack opens and match closing brackets.
- 022. LC 155: Min Stack - Stack + scan for min -> stack of pairs/value+current min -> two stacks.
- 023. LC 150: Evaluate Reverse Polish Notation - Parse manually -> stack operands, apply operator to last two.
- 024. LC 22: Generate Parentheses - Generate all strings and validate -> backtrack with open/close counts.
- 025. LC 739: Daily Temperatures - For each day scan forward -> monotonic decreasing stack of indices.
- 026. LC 853: Car Fleet - Simulate positions -> sort by position desc, stack/track max arrival time.
- 027. LC 84: Largest Rectangle in Histogram - For each bar expand -> precompute boundaries -> monotonic increasing stack.

Binary Search

- 028. LC 704: Binary Search - Linear scan -> classic l/r mid binary search on sorted array.
- 029. LC 74: Search a 2D Matrix - Flatten and scan -> binary search as 1D index -> row then column search.
- 030. LC 875: Koko Eating Bananas - Try speeds linearly -> binary search smallest speed satisfying hours.
- 031. LC 153: Find Minimum in Rotated Sorted Array - Linear min -> binary search compare mid with right side.
- 032. LC 33: Search in Rotated Sorted Array - Linear scan -> binary search identify sorted half each step.
- 033. LC 981: Time Based Key-Value Store - Store list per key -> binary search latest timestamp <= query.
- 034. LC 4: Median of Two Sorted Arrays - Merge arrays -> binary search partition between arrays.

Linked List

- 035. LC 206: Reverse Linked List - Use array/stack -> iterative prev/current reversal -> recursive reversal.
- 036. LC 21: Merge Two Sorted Lists - Collect and sort -> two-pointer merge with dummy node.
- 037. LC 143: Reorder List - Array reorder -> find middle, reverse second half, merge alternately.
- 038. LC 19: Remove Nth Node From End of List - Length then remove -> two pointers with dummy gap n.
- 039. LC 138: Copy List with Random Pointer - Hash map old->new -> interweave copied nodes then split.
- 040. LC 2: Add Two Numbers - Convert to integer -> digit-by-digit linked list with carry.
- 041. LC 141: Linked List Cycle - Hash set visited -> slow/fast Floyd cycle detection.
- 042. LC 287: Find the Duplicate Number - Sort/count/set -> binary search counts -> Floyd cycle on values.
- 043. LC 146: LRU Cache - List scan -> hashmap + doubly linked list for O(1) get/put.
- 044. LC 23: Merge k Sorted Lists - Merge all sort -> pairwise merge -> min-heap of list heads.
- 045. LC 25: Reverse Nodes in k-Group - Count nodes -> reverse each full k block in-place, leave final short block unchanged.

Trees

- 046. LC 226: Invert Binary Tree - DFS swap children -> BFS queue swap level by level.
- 047. LC 104: Maximum Depth of Binary Tree - DFS recursion max depth -> BFS level count.
- 048. LC 543: Diameter of Binary Tree - Compute height for each node -> one DFS returning height and updating diameter.
- 049. LC 110: Balanced Binary Tree - Height each subtree repeatedly -> DFS returns height or imbalance sentinel.
- 050. LC 100: Same Tree - Traverse both -> DFS/BFS compare nodes together.
- 051. LC 572: Subtree of Another Tree - Check equality at every node -> serialize tree with null markers -> DFS match.
- 052. LC 235: Lowest Common Ancestor of a BST - Store paths -> use BST ordering to walk split point.
- 053. LC 102: Binary Tree Level Order Traversal - DFS by depth -> BFS queue level by level.
- 054. LC 199: Binary Tree Right Side View - BFS take last per level -> DFS right-first first node per depth.
- 055. LC 1448: Count Good Nodes in Binary Tree - DFS carry max value on path; count node if value >= max.
- 056. LC 98: Validate Binary Search Tree - Inorder must be sorted -> DFS with low/high bounds.
- 057. LC 230: Kth Smallest Element in a BST - Inorder list -> iterative inorder stop at kth.
- 058. LC 105: Construct Binary Tree from Preorder and Inorder Traversal - Recursive root split in inorder -> hashmap inorder positions.
- 059. LC 124: Binary Tree Maximum Path Sum - DFS max gain; update answer with left+node+right path.
- 060. LC 297: Serialize and Deserialize Binary Tree - Level-order with nulls -> preorder DFS with null markers.

Tries

- 061. LC 208: Implement Trie - Hash set words -> trie nodes with children and end flag.
- 062. LC 211: Design Add and Search Words Data Structure - Store words by length -> trie with DFS branching on '.' wildcard.
- 063. LC 212: Word Search II - Run word search for each word -> build trie and backtrack board once.

Heap / Priority Queue

- 064. LC 703: Kth Largest Element in a Stream - Sort every add -> maintain min-heap of size k.
- 065. LC 1046: Last Stone Weight - Sort repeatedly -> max-heap, pop two largest and push difference.
- 066. LC 973: K Closest Points to Origin - Sort by distance -> max-heap size k -> min-heap all points.
- 067. LC 215: Kth Largest Element in an Array - Sort -> min-heap size k -> quickselect partition.
- 068. LC 621: Task Scheduler - Simulate greedily -> max-heap counts with cooldown queue -> frequency math formula.
- 069. LC 355: Design Twitter - Store tweets per user; merge followee feeds by timestamp using heap.
- 070. LC 295: Find Median from Data Stream - Sort on demand -> two heaps: max lower half, min upper half.

Backtracking

- 071. LC 78: Subsets - Bitmask all subsets -> DFS choose/skip each number.
- 072. LC 39: Combination Sum - DFS choices, reuse same index, stop when sum exceeds target.
- 073. LC 46: Permutations - Track used values -> swap/backtrack in-place.
- 074. LC 90: Subsets II - Sort then DFS, skip duplicate choices at same depth.
- 075. LC 40: Combination Sum II - Sort, use each index once, skip duplicates at same depth.
- 076. LC 79: Word Search - DFS from each matching cell with visited marking/backtracking.
- 077. LC 131: Palindrome Partitioning - DFS cut positions; only recurse on palindrome prefix.
- 078. LC 17: Letter Combinations of a Phone Number - DFS build string from digit-to-letters map.
- 079. LC 51: N-Queens - Backtrack row by row; track columns and diagonals sets.

Graphs

- 080. LC 200: Number of Islands - Scan grid; DFS/BFS flood-fill each unvisited land island.
- 081. LC 133: Clone Graph - DFS/BFS with map original node -> cloned node.

082. LC 695: Max Area of Island - Flood-fill each island and count cells; keep max.
083. LC 417: Pacific Atlantic Water Flow - DFS from each cell -> reverse DFS/BFS from both oceans and intersect.
084. LC 130: Surrounded Regions - Mark border-connected O cells, then flip remaining O to X.
085. LC 994: Rotting Oranges - Multi-source BFS from all rotten oranges by minutes.
086. LC 286: Walls and Gates - Run BFS from every room -> multi-source BFS from all gates.
087. LC 207: Course Schedule - DFS cycle detection -> topological sort by indegree.
088. LC 210: Course Schedule II - Topological sort; return order if all courses processed.
089. LC 261: Graph Valid Tree - Check edges == n-1 and connected via DFS/Union-Find.
090. LC 323: Number of Connected Components in an Undirected Graph - DFS/BFS components -> Union-Find count roots.
091. LC 684: Redundant Connection - Union-Find; first edge whose endpoints already connected is answer.
092. LC 127: Word Ladder - BFS shortest path; use wildcard pattern buckets or neighbor generation.

Advanced Graphs

093. LC 332: Reconstruct Itinerary - Backtrack tickets lexicographically -> Hierholzer DFS with min-heaps.
094. LC 1584: Min Cost to Connect All Points - Build edges then Kruskal MST -> Prim MST without storing all edges.
095. LC 743: Network Delay Time - Bellman-Ford -> Dijkstra shortest paths from source.
096. LC 269: Alien Dictionary - Build graph from first differing chars in adjacent words -> topological sort with cycle check.
097. LC 778: Swim in Rising Water - Binary search time + DFS -> Dijkstra/min-heap on elevation max.
098. LC 787: Cheapest Flights Within K Stops - DFS all paths -> Bellman-Ford k+1 relaxations -> BFS/heap with stops.

1-D Dynamic Programming

099. LC 70: Climbing Stairs - Recursive brute force -> memoization -> bottom-up Fibonacci $O(1)$ space.
100. LC 746: Min Cost Climbing Stairs - Recursion -> DP array -> two rolling variables.
101. LC 198: House Robber - Try all subsets -> DP take/skip -> rolling prev1/prev2.
102. LC 213: House Robber II - Circular case: max(rob houses 0..n-2, rob houses 1..n-1).
103. LC 5: Longest Palindromic Substring - Check all substrings -> expand around centers -> DP palindrome table.
104. LC 647: Palindromic Substrings - DP table -> expand around every center and count.
105. LC 91: Decode Ways - DFS choices -> memo -> bottom-up DP from right to left.
106. LC 322: Coin Change - DFS all combos -> BFS amounts -> DP min coins for each amount.
107. LC 152: Maximum Product Subarray - Brute all subarrays -> track current max/min product because negatives swap roles.
108. LC 139: Word Break - DFS partitions -> memo -> bottom-up DP over string positions.
109. LC 300: Longest Increasing Subsequence - DP $O(n^2)$ -> patience sorting/binary search tails $O(n \log n)$.
110. LC 416: Partition Equal Subset Sum - Backtracking subsets -> 0/1 knapsack DP to target sum/2.

2-D Dynamic Programming

111. LC 62: Unique Paths - DFS grid paths -> DP table -> combinatorics / 1D DP.
112. LC 1143: Longest Common Subsequence - Recursive choices -> 2D DP comparing last/next characters.
113. LC 309: Best Time to Buy and Sell Stock with Cooldown - DFS states -> DP states hold/sold/cooldown.
114. LC 518: Coin Change II - DFS combinations -> 2D/1D DP count ways by coin order.
115. LC 494: Target Sum - DFS +/- signs -> memo -> transform to subset-sum DP.
116. LC 97: Interleaving String - DFS i,j -> memo -> 2D DP matching chars from s1/s2.
117. LC 329: Longest Increasing Path in a Matrix - DFS from each cell with memoized path length.
118. LC 115: Distinct Subsequences - DFS match/skip -> 2D DP count subsequences.
119. LC 72: Edit Distance - Recursive operations -> 2D DP insert/delete/replace.
120. LC 10: Regular Expression Matching - Backtracking -> memo/DP over string and pattern, handle '*' branch.
121. LC 312: Burst Balloons - Brute burst orders -> interval DP choose last balloon in interval.

Greedy

122. LC 53: Maximum Subarray - Brute all subarrays -> Kadane keep best ending here and global best.
123. LC 55: Jump Game - DP reachable -> greedy farthest reachable index.
124. LC 45: Jump Game II - BFS levels -> greedy current window end and farthest next jump.
125. LC 134: Gas Station - Try each start -> greedy reset start when tank goes negative.
126. LC 846: Hand of Straights - Sort cards; greedily start groups using count map/min-heap.
127. LC 1899: Merge Triplets to Form Target Triplet - Filter triplets not exceeding target; OR/check each target coordinate found.
128. LC 763: Partition Labels - Last index map; extend partition until current index reaches farthest last occurrence.
129. LC 678: Valid Parenthesis String - DP/stack -> greedy range of possible open counts.

Intervals

130. LC 57: Insert Interval - Append then merge -> scan before/overlap/after inserted interval.
131. LC 56: Merge Intervals - Sort by start then merge overlapping ranges into result.
132. LC 435: Non-overlapping Intervals - DP -> sort by end and greedily remove intervals that overlap.
133. LC 252: Meeting Rooms - Sort intervals; any adjacent overlap means cannot attend all.
134. LC 253: Meeting Rooms II - Min-heap end times -> sweep line start/end events count max rooms.
135. LC 1851: Minimum Interval to Include Each Query - Brute check each query -> sort intervals/queries, min-heap valid interval sizes.

Math & Geometry

136. LC 48: Rotate Image - Use extra matrix -> transpose then reverse each row -> layer-by-layer 4-way swap.
137. LC 54: Spiral Matrix - Visited simulation -> shrinking top/bottom/left/right boundaries.
138. LC 73: Set Matrix Zeroes - Use sets rows/cols -> use first row/col as markers $O(1)$ space.
139. LC 202: Happy Number - Hash set seen sums -> Floyd cycle detection on digit-square function.
140. LC 66: Plus One - Convert to int -> traverse digits from right with carry.
141. LC 50: Pow(x, n) - Repeated multiply -> fast exponentiation by squaring, handle negative n.
142. LC 43: Multiply Strings - Convert to int not allowed -> grade-school digit multiplication array.
143. LC 2013: Detect Squares - Store point counts; for query, try same-y points as diagonals/sides and multiply counts.

Bit Manipulation

144. LC 136: Single Number - Hash counts -> XOR all numbers cancels pairs.
145. LC 191: Number of 1 Bits - Loop all bits -> n & (n-1) repeatedly removes lowest set bit.
146. LC 338: Counting Bits - Count each number bits -> DP: bits[i] = bits[i>>1] + (i&1).
147. LC 268: Missing Number - Sort/sum formula -> XOR indices and values.
148. LC 190: Reverse Bits - Shift result left and append n&1 for 32 iterations.
149. LC 371: Sum of Two Integers - Use XOR for sum without carry, AND<<1 for carry until zero.
150. LC 7: Reverse Integer - Pop digits and push into result; check 32-bit overflow before append.